

变异测试背景

变异测试的产生

两个测试人员关心的问题：

- 如何编写能够暴露缺陷的测试用例 → 如何引导测试
- 如何评估测试套件的质量（提升测试可信度） → 如何评估测试

变异测试的产生：

- 模拟缺陷，量化缺陷检测能力，扮演测试有效性的指示器
- 模拟：变异产生错误版本，模拟探测 Bug 的过程
- 量化：变异得分（变异杀死率）

研究现状

变异测试：

- 从缺陷的角度评估测试用例/套件（Fault-based Testing）
- 在学术界和工业界都得到了广泛关注

 按出版物分

 按时间分布

变异分析与变异测试

变异分析（Mutation Analysis） → Research

- 自动化“变异”源程序以得到源程序语义变体，并对其进行优化和分析的过程 → 自动化生成人工缺陷
- 语义变体：语法上、语义上与源程序都不相同 → 变异体
- “变异”：程序变换规则 → 变异算子

变异测试（Mutation Testing） → Practice

- 将变异体视作测试目标（Test Goal）
- 利用变异分析的结果来支持、评估、引导软件测试的过程

变异体

变异体（Mutant）：

- 基于一定的语法（Syntax）变换规则，通过对源程序进行程序变换（Program Transformation）得到的一系列变体

假设：

- 源程序不包含缺陷 → 现有的测试套件没有发现缺陷
- 变异体表达了某种缺陷 → 人工缺陷（Artificial Defect）

源程序	变异体
<pre>public void foo(int x, int y) { if (x > y) return x; else return y; }</pre>	<pre>public void foo(int x, int y) { if (x > y) return x; else return x; // A Bug }</pre>

变异得分

变异得分 (Mutation Score) :

- 变异测试对测试套件错误检测能力的量化
- 杀死与存活: 变异体是否导致某个测试用例运行失败; 测试用例是否“检测”到某个变异体
 - 检测到: 变体被杀死 (Killed)
 - 未检测到: 变体存活 (Survived)

计算公式:

$$score = \frac{mut_k}{mut_s + mut_k} \times 100\%$$

变异杀死的条件

缺陷传播模型 RIPR 和 PIE:

- RIPR: Reachability, Infection, Propagation Revealability
- PIE: Execution, Infection, Propagation

解释:

1. R & E: 缺陷所在的位置可以被执行到
2. I: 缺陷的执行影响了程序的状态
3. P & PR: 程序状态的影响传播到了输出

杀死条件 (Mutant Killing Condition) :

- 受程序行为 (Program Behavior) 的定义影响
- 程序行为:
 - (一般定义) 可观测的程序输出: 待测程序输出到标准输出的内容, 或者被测试用例添加断言 (Assert) 的部分 → Propagation
 - 程序的中间状态: 程序执行某些操作后处在的状态 (Program State) → Infection

变异分类

根据杀死条件的不同, 变异可以分为三种:

- Weak mutation: 变异导致紧随变化位置的程序状态发生了改变 (R & E)
- Firm Mutation: 变异导致远离变化位置的程序状态发生了改变 (I)
- Strong Mutation: 变异导致程序的输出发生了变化 (P & PR)

Weak Mutation → Firm Mutation → Strong Mutation。

变异的要求变高，变异体质量提升。

变异体的分类

- 有效变异体
- 夭折变异体 (Stillborn Mutant)
- 冗余变异体 (Redundant Mutant)
 - 等价变异体 (Equivalent Mutant)
 - 重复变异体 (Duplicated Mutant)
 - 蕴含变异体 (Subsumed Mutant)

等价变异体

等价：对于待测程序 P 的变体 mut ，如果 mut 和 P 的语法不同、语义相同，则称 mut 是 P 的等价变异体。

语义相同：对于给定输入，两个程序总能给出相同的输出。

源程序	变异体
<pre>public void foo(int a) { int abs = 0; if (a > 0) { abs = a; } else { abs = -a; } return abs; }</pre>	<pre>public void foo(int a) { int abs = 0; if (a >= 1) // mutant here { abs = a; } else { abs = -a; } return abs; }</pre>
<pre>public void testFoo() { assert foo(1) == 1; }</pre>	<pre>public void testFoo() { assert foo(1) == 1; }</pre>
PASS	SURVIVE

重复变异体

对于待测程序 P 的两个变体 mut_1 和 mut_2 ，如果 mut_1 和 mut_2 语义相同，则 mut_1 和 mut_2 互为重复：

源程序	变异体 mut_1	变异体 mut_2
<pre> public void foo(int a) { int abs = 0; if (a > 0) { abs = a; } else { abs = -a; } return abs; } </pre>	<pre> public void foo(int a) { int abs = 0; if (a > 1) { // mutant here abs = a; } else { abs = -a; } return abs; } </pre>	<pre> public void foo(int a) { int abs = 0; if (a >= 2) { // mutant here abs = a; } else { abs = -a; } return abs; } </pre>
<pre> public void testFoo() { assert foo(1) == 1; } </pre>	<pre> public void testFoo() { assert foo(1) == 1; } </pre>	<pre> public void testFoo() { assert foo(1) == 1; } </pre>
PASS	KILL	KILL

蕴含变异体

对于两个变体 mut_1 和 mut_2 ，如果任何杀死 mut_1 的测试用例 t 都可以杀死 mut_2 ，则称 mut_1 蕴含 mut_2 。

源程序	变异体 mut_1	变异体 mut_2
<pre> public void foo(int a) { int abs = 0; if (a > 0) { abs = a; } else { abs = -a; } return abs; } </pre>	<pre> public void foo(int a) { int abs = 0; if (a >= 3) { // mutant here abs = a; } else { abs = -a; } return abs; } </pre>	<pre> public void foo(int a) { int abs = 0; if (a >= 4) { // mutant here abs = a; } else { abs = -a; } return abs; } </pre>
<pre> public void testFoo() { assert foo(1) == 1; } </pre>	<pre> public void testSym() { assert foo(x) == x; } </pre>	<pre> public void testSym() { assert foo(x) == x; } </pre>
	KILL $mut_1 : x \in \{1, 2\}$	KILL $mut_2 : x \in \{1, 2, 3\}$

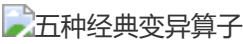
变异算子

变异算子 (Mutation Operator) :

- 一系列语法变换规则 (Syntactic Transformation Rule)
- 变异 (Mutate) 的依据, 反映了测试人员关注的缺陷种类

基本形式:

- 对程序的源代码进行变换 (Source Code Level)
- 对程序的编译结果 (中间表示) 进行变换, 例如: 针对 Java Bytecode 的程序变换算子
- 元变异 (Meta-mutation)



基础假设

变异测试为什么有效?

- 假设 1: 缺陷是简单的、可模拟的
- 假设 2: 缺陷可叠加
- 假设 3: 缺陷检测有效性

假设 1: 缺陷是简单的、可模拟的

变异体能够模拟测试人员关注的缺陷类型, 即: 最常出现的缺陷类型;

能够在变异测试中暴露这些人工缺陷的测试套件一定能检测出待测程序中潜在的同类缺陷

老练程序员假设: 一个老练程序员编写的错误程序与正确程序相差不大

假设 2: 缺陷可叠加

- 复杂变异体可以通过耦合简单变异体得到
- 能够杀死简单变异体的测试用例可以杀死复杂变异体
- 结合 RIPR/PIE 模型思考: 该假设是否总是正确?

源程序	简单变异体	简单变异体
<pre>public void foo(int a) { int abs = 0; if (a > 0) { abs = a; } else { abs = -a; } return abs; }</pre>	<pre>public void foo(int a) { int abs = 0; if (a >= -1) { //mutant here abs = a; } else { abs = -a; } return abs; }</pre>	<pre>public void foo(int a) { int abs = 0; if (a > 0) { abs = a; } else { abs = a; // mutant here } return abs; }</pre>

源程序	简单变异体 mut_1	简单变异体 mut_2
<pre>public void testFoo() { assert foo(1) == 1; }</pre>	<pre>public void testFoo1() { assert foo(-1) == 1; }</pre>	<pre>public void testFoo() { assert foo(-2) == 2; }</pre>
	KILL $mut_1 : x \in \{1, 2\}$	KILL $mut_2 : x \in \{1, 2, 3\}$

$mut_1 + mut_2 =$ 复杂变异体 mut_3 :

```
1 public void foo(int a) {
2     int abs = 0;
3     if (a >= -1) { // mutant here
4         abs = a;
5     } else {
6         abs = a; // mutant here
7     }
8     return abs;
9 }
```

- testFoo1(): KILL
- testFoo2(): KILL

假设 3：缺陷检测有效性

- 缺陷检测能力是测试用例最重要的属性
- 变异得分能够有效地反映测试用例/套件的错误检测能力
- 传递性假设：能够杀死变异体（暴露人工缺陷）的测试用例也能有效发现真实世界的缺陷

变异测试过程



- 变异分析：1, 2, 3, 5, 6
- 变异应用：4, 7, 10

Step 1：变异体筛选

变异筛选/约减 (Reduction)：

- 对变异体进行筛选 (Mutant Selection)：从生成的变体集合中选出一个子集
- 通过变异算子筛选 (Selective Mutation)：定义一系列变异算子，选取其中的一部分构成子集

研究内容：

- 变异算子的定义
- 变异算子的选取/约减策略

变异算子定义

变异体约减策略

Step 2: 变异体生成

变异体生成：将选中的变异体（算子）实例化

基本方式：为每个变体构建一个单独的源文件（Source File）

缺陷：运行时间较长，会产生较大的额操作开销

研究内容：变异体生成技术

- 元变异（Meta-mutation）
- 基于字节码操作（Bytecode Manipulation）
- 热替换（Hot-Swap, Just-In-Time）

Step 3: 变异体优化

内容：去除有等价和无效变异体

目的：

- 减少后续执行变异体阶段的开销
- 提高变异得分的可信度

基本形式：通过静态分析的方式，识别并移除有问题的变异体

Step 5: 变异体执行

特征：变异测试过程中最昂贵的阶段

基本形式：假设一个待测程序 P 有 n 个变异体和一个包含 m 个测试用例的测试套件，则执行变异体的最大次数为 $n \times m$ 。

研究内容：针对执行优化的两大场景优化执行过程

- 场景 A：计算变异得分
- 场景 B：计算变异体矩阵（Mutant Matrix）

Step 6: 变异得分计算

计算被杀死的变异体数量，进而得到变异得分、量化测试的充分性

研究内容：

- 变异杀死的条件：确定一个变异体是否被杀死
- 测试预言的生成：如何生成更多、更好的测试预言来杀死更多的变异体、提高变异得分

变异杀死的条件：

- 核心：定义程序行为（Program behavior）
- Weak, Firm, Strong Mutation
- 代表性工作：
 - 针对确定性系统：定义新的系统级行为

- 针对非确定性系统：定义、模拟程序规格

变异测试应用

- 评估作用：利用变异得分度量测试的充分性
- 引导作用：利用变异测试/分析的结果来引导测试过程
- 传统应用：应用于确定性系统
 - 测试生成 (Test Generation)
 - 预言生成 (Oracle Problem)
 - 测试优化 (排序 & 选择)
 - Debug 引导 (缺陷定位 & 自动修复)
- 变异 & AI：应用于非确定性系统 (DNN)

用例与预言生成

变异辅助 Debug

利用变异自动为有缺陷程序推荐补丁：

- Debug 的两个阶段：
 1. 定位：判断缺陷所在位置、分析可疑的缺陷特征
 2. 修复：着手修复缺陷本身
- 自动 Debug 的挑战：
 - 缺少缺陷版本作为试验评估的数据集
 - 变异分析的自身特性为自动 Debug 提供了数据支持

缺陷定位技术：

- 抽象测试轨迹，并利用这些轨迹计算程序组件的可疑度 (Suspiciousness)
- 以可疑度作为指标，挑选出最有可能的缺陷组件 (Faulty Component)

自动修复技术：

- 根据一定的语法规则转换缺陷程序为正确版本
- 结合变异：是否可以逆向利用变异过程进行自动修复？

技术挑战：全局变异分析会带来较大的开销，降低整个流程的效率

解决方案：

- 利用缺陷定位技术进行可疑度排序
- 高可疑组件即为最可能的缺陷位置
- 按照可疑度从高到低的顺序对组件进行变异

AI 测试 TCP

通过变异分析进行 DNN 测试输入排序：

- 变异分析 + 测试排序 + AI 测试
- TCP for DNN：回答“谁应该最先被打标签？”

- MA for DNN: 模型的变异、输入的变异
- 核心思路: 能够杀死最多变异模型的, 且变异后能够产生最多预测结果的测试输入最有可能暴露 DNN 模型中存在的缺陷



总结

